Sistema de Gestão de Inventário e Vendas de Supermercado

Relatório Técnico de Engenharia de Requisitos (Iteração 01)

Tomás Bastos (1251506), Igor Almeida (1251054), Guilherme Asevedo (1251002), and Rui Silva (1251443)

Instituto Superior de Engenharia do Porto (ISEP)
Licenciatura em Engenharia de Telecomunicações e Informática
Unidade Curricular: Fundamentos de Desenvolvimento de Software
Grupo de Trabalho: 1DB_3

Resumo Este relatório detalha a fase inicial de análise do projeto de software para a gestão de um supermercado moderno. Através de uma abordagem rigorosa de Engenharia de Requisitos, exploramos o domínio do problema, identificamos os atores intervenientes e mapeamos as funcionalidades críticas que o sistema deve suportar. O documento segue as diretrizes da Análise Orientada a Objetos (OOA), apresentando User Stories, Requisitos Funcionais e Não-Funcionais sob o modelo FURPS+, um Glossário terminológico, Modelos de Casos de Uso com especificações detalhadas, Diagramas de Sequência de Sistema (SSD) e o Modelo de Domínio conceptual. O objetivo principal é garantir que a fundação do software seja sólida, modular e alinhada com as melhores práticas de C++ contemporâneo, preparando o terreno para a implementação baseada no padrão MVC.

Key words: Engenharia de Requisitos, C++, OOA, UML, Modelo de Domínio, FURPS+, Casos de Uso, Supermercado

1 Introdução

1.1 Contextualização do Problema

No mercado retalhista contemporâneo, a eficiência operacional de um supermercado depende criticamente da precisão com que o inventário é gerido e as vendas são processadas. A ausência de um sistema integrado conduz a problemas recorrentes, como a rotura de stock (*out-of-stock*), o desperdício de produtos perecíveis e a opacidade na análise de rentabilidade.

Este projeto propõe o desenvolvimento de um "Sistema de Gestão de Inventário e Vendas de Supermercado" robusto, desenhado especificamente para ambientes CLI (*Command Line Interface*), priorizando a lógica de negócio modular e a persistência de dados fidedigna. O sistema deve servir dois perfis principais: o Gestor, focado na estratégia e aprovisionamento, e o Caixa, focado na operação transacional.

1.2 Objetivos do Projeto

Os objetivos primordiais deste desenvolvimento são:

- **Gestão de Inventário Inteligente:** Monitorização em tempo real de níveis de stock com alertas de reposição.
- **Agilização do Ponto de Venda (POS):** Interface simplificada para registo de vendas e cálculo de impostos (IVA).
- **Fidelização de Clientes:** Implementação de um programa de pontos para retenção de consumidores.
- **Análise Financeira:** Relatórios automáticos de lucro baseados na diferença entre preços de custo (Wholesale) e venda (Retail).
- **Escalabilidade e Qualidade:** Utilização do padrão MVC (*Model-View-Controller*) para garantir código limpo e testável.

1.3 Motivação da Equipa 1DB_3

A equipa 1DB_3 encara este projeto como uma oportunidade de aplicar os conceitos de programação orientada a objetos (POO) em C++ num cenário do mundo real. A transição da análise para o código exige um rigor analítico que este documento pretende satisfazer, seguindo os padrões académicos mais elevados do ISEP.

2 Estado da Arte e Análise de Domínio

2.1 Evolução dos Sistemas de Gestão de Retalho

Tradicionalmente, a gestão de pequenas mercearias era feita manualmente em livros de registo. Com a digitalização, surgiram os sistemas de POS que integram leitores de códigos de barras e bases de dados relacionais. Embora este projeto utilize ficheiros de texto/CSV para persistência (em conformidade com os requisitos da UC), a arquitetura interna emula sistemas profissionais através de contentores (*Containers*) e gestão de exceções customizadas.

2.2 Análise do Mercado Alvo

O sistema é direcionado para supermercados de média dimensão que necessitam de uma ferramenta leve, rápida e que possa correr em hardware com recursos limitados, mantendo a integridade dos dados e a segurança de acesso.

3 Metodologia de Engenharia de Requisitos

Seguindo as diretrizes das aulas teóricas (especialmente a aula de 26/03), a equipa adotou uma metodologia iterativa e incremental:

1. **Levantamento:** Identificação de necessidades através da análise da especificação do projeto.
2. **Especificação de User Stories:** Redação de requisitos centrados no utilizador.
3. **Modelação de Domínio:** Identificação de entidades e relações sem enviesamento técnico.
4. **Modelação de Processos:** Criação de Casos de Uso e SSDs para definir fluxos de mensagens.
5. **Validação:** Revisão cruzada entre os membros da equipa para detetar ambiguidades.

4 Especificação de Requisitos

4.1 User Stories (US)

As User Stories representam os requisitos do ponto de vista de quem utiliza o sistema, permitindo focar no valor de negócio.

Tabela 1: Catálogo de User Stories

ID	User Story	Critérios de Aceitação
US01	Como **Utilizador**, quero fazer login para aceder às funções do meu perfil.	- Validação de credenciais contra ficheiro seguro. - Redirecionamento para o dashboard correto (Gestor vs Caixa).
US02	Como **Gestor**, quero adicionar um produto ao inventário.	- Verificação de SKU duplicado. - Validação de preços positivos.
US03	Como **Gestor**, quero remover produtos obsoletos.	- Confirmação antes da eliminação definitiva.
US04	Como **Gestor**, quero editar preços para responder à inflação.	- Atualização imediata em todos os cálculos de venda futuros.
US05	Como **Gestor**, quero definir um limite mínimo de stock.	- Sistema deve sinalizar produtos abaixo do limite.
US06	Como **Caixa**, quero adicionar itens a um carrinho de compras.	- Validação de stock disponível antes de adicionar. - Visualização parcial do total.
US07	Como **Caixa**, quero aplicar descontos de fidelização.	- Verificação de saldo de pontos do cliente. - Abate de pontos após conclusão.
US08	Como **Gestor**, quero gerir fornecedores.	- CRUD de contactos de fornecedores.
US09	Como **Gestor**, quero ver o lucro líquido diário.	- Relatório calculando (Preço Venda - Preço Custo) x Qtd.
US10	Como **Utilizador**, quero pesquisar produtos por categoria.	- Listagem filtrada e ordenada alfabeticamente.

4.2 Requisitos Funcionais (RF)

Os requisitos funcionais detalham o comportamento específico do software.

- **RF01 - Autenticação:** O sistema deve suportar autenticação por Username e Password.
- **RF02 - RBAC:** O sistema deve implementar controlo de acesso baseado em funções (Gestor e Caixa).
- **RF03 - Gestão de Inventário:** O sistema deve permitir criar, ler, atualizar e apagar (CRUD) produtos.
- **RF04 - Pesquisa Avançada:** O sistema deve permitir procurar produtos por SKU, Nome ou Categoria.
- **RF05 - Vendas (POS):** O sistema deve suportar a criação de sessões de venda, adição de múltiplos itens e cálculo de IVA (23%, 13%, 6%).
- **RF06 - Gestão de Fornecedores:** O sistema deve manter um registo de fornecedores associados a categorias de produtos.
- **RF07 - Encomendas (Procurement):** O sistema deve permitir gerar ordens de compra para fornecedores, aumentando o stock automaticamente após a receção.
- **RF08 - Fidelização:** O sistema deve registar clientes por número de telemóvel e acumular 1 ponto por cada 10€ gastos.
- **RF09 - Relatórios:** Geração de ficheiros .txt com o resumo de vendas diárias e balanço de stock.
- **RF10 - Notificações:** Alertas automáticos no dashboard do gestor para produtos com stock crítico ou validade próxima.

4.3 Requisitos Não-Funcionais (FURPS+)

Utilizamos o modelo FURPS+ para garantir a qualidade holística do sistema.

F - Functionality (Funcionalidade)

- **Persistência:** Os dados devem ser guardados em ficheiros CSV, garantindo que a informação não se perde ao encerrar o programa.
- **Segurança:** As passwords não devem ser visíveis durante a digitação no terminal (sempre que possível).

U - Usability (Usabilidade)

- **Interface CLI:** Menus intuitivos com atalhos numéricos (ex: "1. Vendas", "2. Inventário").
- **Consistência:** O formato de datas (DD-MM-AAAA) e moedas (0.00€) deve ser uniforme em todo o sistema.

R - Reliability (Fiabilidade)

- **Robustez:** O sistema não deve terminar abruptamente (*crash*) se o utilizador introduzir letras num campo numérico. Deve usar *Exceptions* para lidar com entradas inválidas.
- **Disponibilidade:** O arranque do sistema deve demorar menos de 2 segundos.

P - Performance (Desempenho)

- **Tempo de Resposta:** A pesquisa de um produto num catálogo de 10.000 itens deve ser inferior a 100ms.
- **Eficiência de Memória:** O uso de memória deve ser otimizado através do uso de referências e ponteiros inteligentes (`smart pointers`) onde aplicável.

S - Supportability (Suportabilidade)

- **Portabilidade:** O código deve ser compilável em Linux e Windows usando o standard C++20.
- **Manutenibilidade:** Seguimento estrito do padrão MVC para separação de responsabilidades.

+ - Outros (Restrições de Design)

- **Linguagem:** Todo o código fonte e comentários devem estar em Inglês (ou conforme definido na norma da equipa), mas a interface de utilizador deve ser clara.
- **Modularidade:** Uso obrigatório de classes "Container" para gerir coleções de objetos.

5 Modelação de Casos de Uso

5.1 Diagrama de Casos de Uso

O diagrama abaixo ilustra a fronteira do sistema e a interação entre os atores e as principais funcionalidades.

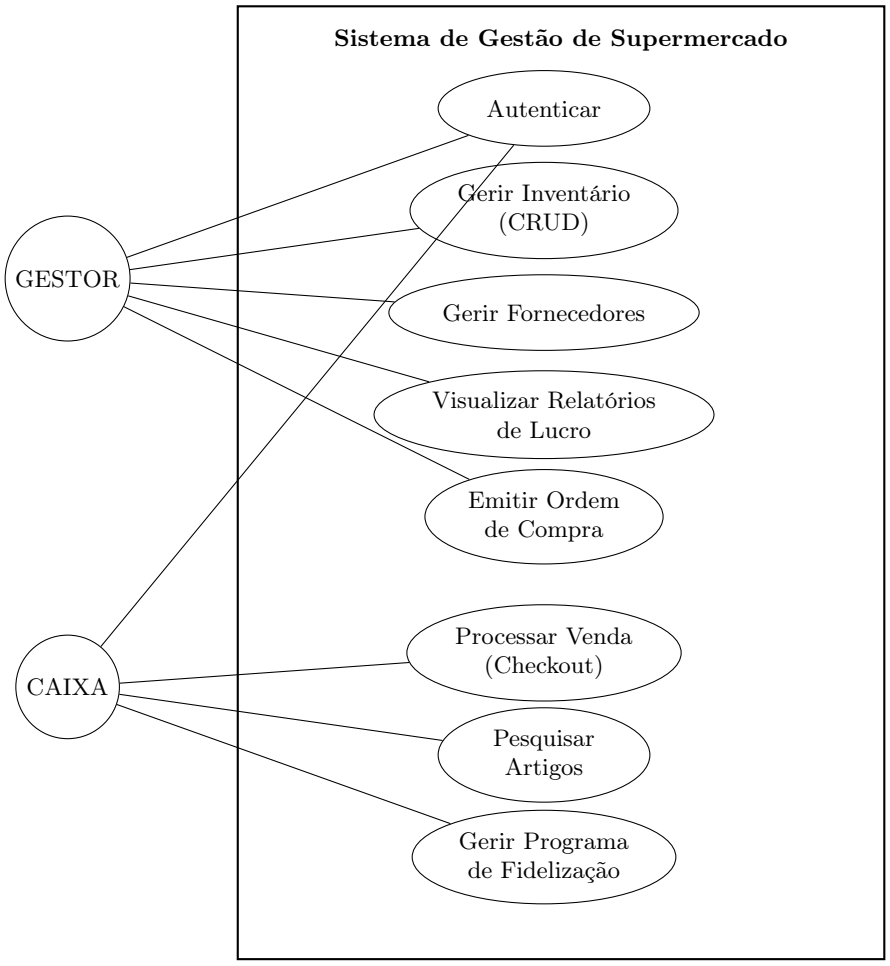

Figura 1. Diagrama UML de Casos de Uso - Visão Geral.

5.2 Especificações Detalhadas de Casos de Uso

Abaixo apresentam-se as especificações detalhadas para os casos de uso críticos, definindo os fluxos de sucesso e as condições de contorno.

UC1 - Autenticar Utilizador

Atributo	Descrição
Ator Principal	Gestor ou Caixa
Interessados	Segurança do sistema; Equipa de Auditoria.
Pré-condições	O sistema está no ecrã de boas-vindas.

Fluxo Principal	1. O utilizador solicita o login. 2. O sistema pede o nome de utilizador. 3. O utilizador introduz o nome. 4. O sistema pede a palavra-passe. 5. O utilizador introduz a palavra-passe. 6. O sistema valida as credenciais no contentor de utilizadores. 7. O sistema identifica o perfil do utilizador. 8. O sistema redireciona para o menu correspondente.
Fluxos Alternativos	3a. O utilizador cancela a operação: regressa ao ecrã inicial. 6a. Credenciais inválidas: o sistema exibe erro e permite 3 tentativas.
Pós-condições	Sessão ativa com permissões de acesso definidas.

UC5 - Processar Venda (Checkout)

Atributo	**Descrição**
Ator Principal	Caixa
Interessados	Cliente (recebe recibo); Gestor (vê stock descer).
Pré-condições	Caixa autenticado; Carrinho de compras vazio.
Fluxo Principal	1. O Caixa inicia uma nova venda. 2. O Caixa introduz o SKU de um artigo. 3. O Sistema valida o SKU e a existência de stock. 4. O Sistema adiciona o item ao carrinho e calcula o IVA parcial. 5. Repetem-se os passos 2-4 até terminar. 6. O Caixa solicita o número de telemóvel do cliente (opcional). 7. O Sistema calcula o total e os pontos a ganhar. 8. O Caixa confirma o pagamento. 9. O Sistema gera o recibo .txt, atualiza o stock e grava a transação.
Fluxos Alternativos	3a. SKU não encontrado: Sistema informa e permite nova entrada. 3b. Stock insuficiente: Sistema alerta sobre a quantidade disponível. 8a. Venda cancelada: Sistema limpa o carrinho e não altera stock.
Pós-condições	Venda registada no histórico; Stock decrementado; Recibo emitido.

6 Diagramas de Sequência de Sistema (SSD)

Os SSDs detalham as trocas de mensagens entre os atores e o sistema, tratando-o como uma entidade única ("caixa preta").

6.1 SSD: UC2 - Adicionar Novo Produto

Este cenário descreve um Gestor a registar um novo item no inventário.

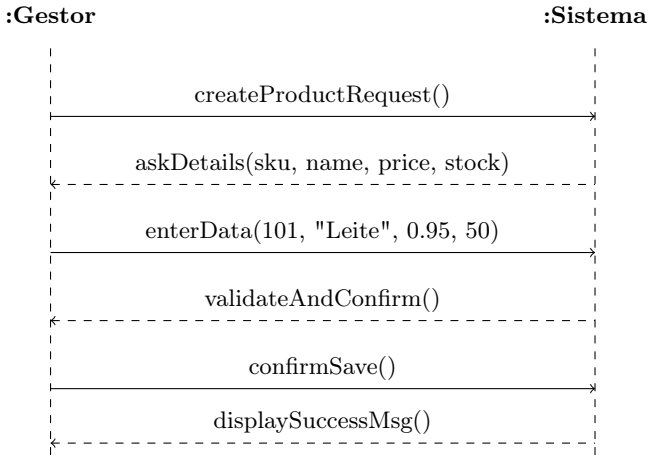

Figura 2. SSD para a criação de um produto com sucesso.

6.2 SSD: UC5 - Fluxo de Venda com Fidelização

Este cenário descreve um Caixa a realizar uma venda onde o cliente é identificado.

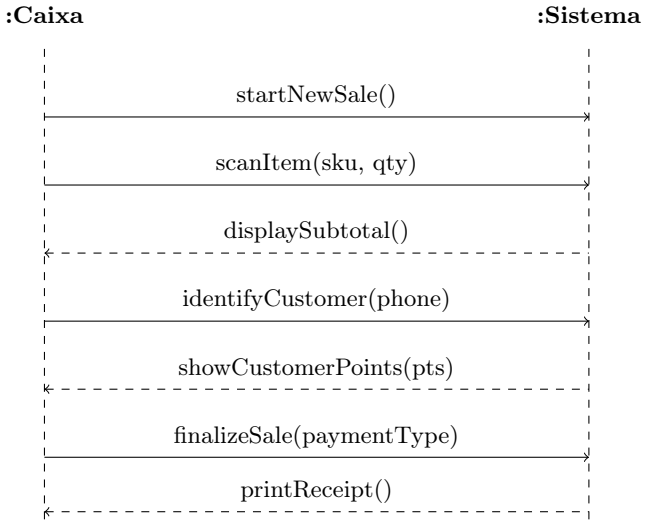

Figura 3. SSD para uma venda completa com identificação de cliente.

7 Modelo de Domínio (OOA)

O Modelo de Domínio representa as entidades conceptuais do "mundo real" do supermercado e as suas relações. Seguindo as regras da aula teórica, não incluímos tipos de dados técnicos ou IDs de bases de dados.

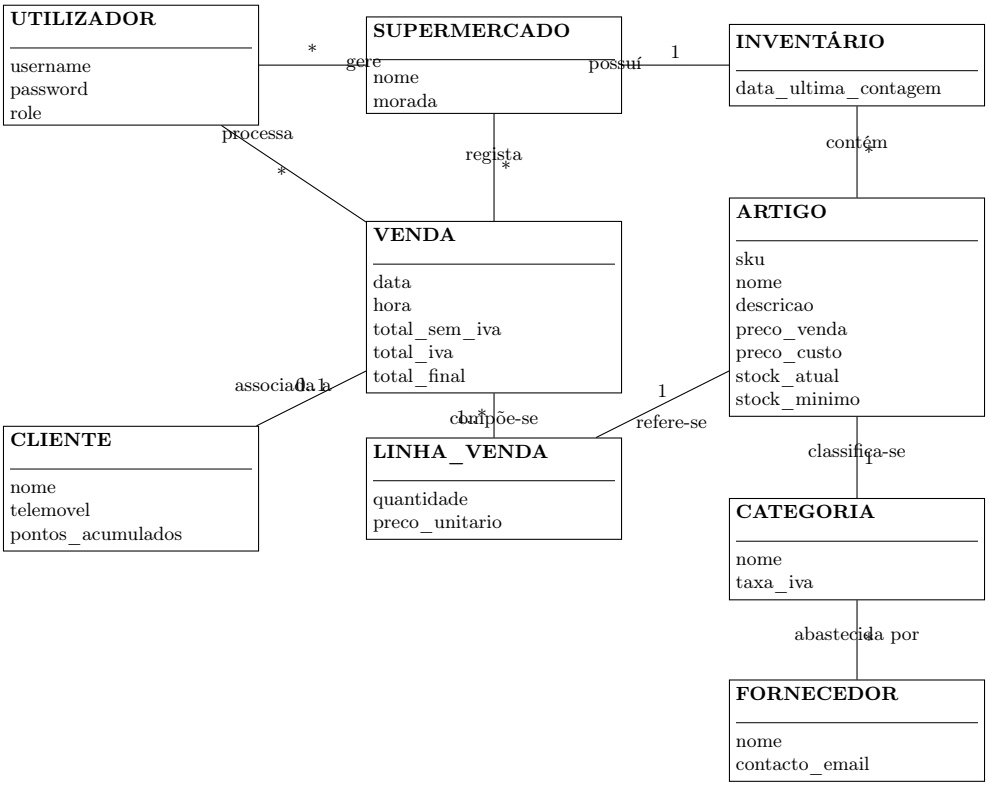

Figura 4. Modelo de Domínio Conceptual (OOA) - Notação UML.

7.1 Dicionário de Classes do Domínio

- **SUPERMERCADO:** A entidade raiz que centraliza toda a operação.
- **UTILIZADOR:** Representação genérica de quem opera o sistema (Gestor/Caixa).
- **ARTIGO:** Representa um produto físico vendável. O SKU (*Stock Keeping Unit*) é o seu identificador universal no negócio.
- **CATEGORIA:** Agrupamento de artigos (ex: Laticínios, Limpeza). Define a taxa de IVA aplicável.
- **VENDA:** Registo de uma transação comercial completa.
- **LINHA_VENDA:** Classe associativa que permite registar que um artigo foi vendido numa determinada quantidade numa venda específica.
- **CLIENTE:** Entidade que participa no programa de fidelização.
- **FORNECEDOR:** Parceiro externo que providencia stock para determinadas categorias.

8 Glossário de Termos

Para garantir que não existem ambiguidades entre a equipa de desenvolvimento e os utilizadores finais, definimos os seguintes termos:

Ator: Qualquer entidade externa que interage com o sistema (Humano ou outro Sistema).
CRUD: Sigla para *Create, Read, Update, Delete*. Operações básicas de gestão de dados.
Dashboard: Ecrã principal de um utilizador após o login, contendo o resumo das suas funções.
IVA: Imposto sobre o Valor Acrescentado. Calculado com base na categoria do produto.
MVC: *Model-View-Controller*. Padrão de arquitetura de software que separa a lógica de dados, a interface e o controlo de fluxo.
POS: *Point of Sale*. O ponto onde a transação de venda ocorre.
Procurement: Processo de aquisição de bens junto de fornecedores.
SKU: *Stock Keeping Unit*. Código alfanumérico único para cada produto diferente no inventário.
SSD: *System Sequence Diagram*. Diagrama que mostra os eventos de entrada e saída do sistema.
Wholesale Price: Preço de custo (preço por grosso) pago ao fornecedor.
Retail Price: Preço de venda ao público praticado no supermercado.

9 Plano de Testes e Qualidade

A verificação e validação do sistema serão feitas através de várias camadas:

9.1 Testes Unitários

Cada classe do Modelo (`Product`, `User`, `Customer`) será testada individualmente para garantir que os *getters*, *setters* e lógica interna (ex: cálculo de pontos) funcionam como esperado. Será utilizada a biblioteca `GoogleTest`.

9.2 Testes de Integração

Garantir que os Contentores (`ProductContainer`) conseguem ler e escrever nos ficheiros CSV sem corromper dados.

9.3 Testes de Aceitação

Baseados nos critérios de aceitação definidos nas User Stories. O sistema deve ser capaz de completar um fluxo de venda do início ao fim sem erros.

10 Riscos e Mitigação

1. **Risco:** Perda de dados por encerramento forçado do PC. **Mitigação:** Implementar gravação automática em disco após cada transação crítica.
2. **Risco:** SKU duplicado por erro humano. **Mitigação:** Validação rigorosa na camada *Controller* antes de permitir a inserção no *Model*.

11 Conclusão

A conclusão desta Iteração 01 marca o fim da fase de análise. Através da decomposição do sistema em Requisitos, Casos de Uso e Modelos de Domínio, a equipa 1DB_3 obteve uma compreensão profunda das necessidades do supermercado.

A arquitetura proposta, focada na modularidade e no uso de padrões de desenho, garante que a fase de implementação (Iteração 02) será guiada por especificações claras, reduzindo a necessidade de retrabalho e aumentando a robustez do software final. O sistema está agora pronto para ser desenhado sob a perspetiva de classes de software em C++.

Referências

1. Docentes de FSOFT: Resumo da Aula Teórica de 26/03 - Engenharia de Requisitos e Modelação OO. ISEP (2026).
2. Larman, C.: Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design. Prentice Hall (2004).
3. Stroustrup, B.: The C++ Programming Language, 4th Edition. Addison-Wesley (2013).
4. ISO/IEC/IEEE 29148:2018: Systems and software engineering — Life cycle processes — Requirements engineering.
5. Exemplo de Projeto "School Application": Material de Apoio à UC FSOFT. DEE-ISEP (2025).